LEAP Demo Script

LeMarketJames · Business Requirements review · 30 minutes, live, on the local Docker stack

How to read this script. DRIVER **Amara** operates the screen and terminal. NARRATOR **Johan** says the lines in *italics*; say them in your own words, but make every point. ARCHITECT **Aditya and Wali** answer "why this design" and "where does it live". SCRIBE **Joseph and Will** write down the reviewer's picks, the numbers to compare, and anything not fully shown. Dark boxes are **short command names you type** into a terminal where `source ~/lmj-demo/demo.sh` has been run; light boxes show the expected output. **Never copy commands out of this PDF:** PDFs break underscores and line breaks when copied. Every command lives in `~/lmj-demo/demo.sh`; `demo_help` lists them. Every command was tested on the stack on 23 Oct. The team still rehearses each one so the timing and talking are smooth.

Agreed with the instructor (Narrator says this in the first minute)

- The market is **US stocks only**: 51 US equities in USD. There is no UK, Indian, FX or crypto trading.
- The demo runs on **our local stack**: four Spring Boot services, one PostgreSQL database and Kafka in Docker on this laptop. There's no cloud deployment.
- The quote outage test **breaks the whole price feed**, not one stock.

Run of show

Time	Act	Requirements shown
0:00–1:00	0. Opening	Scope, what's live, roles
1:00–4:00	1. A brand-new client	BR-01, BR-03 (mentioned)
4:00–11:00	2. Trading as Client A	BR-04, 05, 07, 08, 09, 10, 11, 12, 13, BR-02 (isolation), BR-18
11:00–14:00	3. Trace a trade the reviewer picks	BR-15, BR-14
14:00–22:00	4. Break it	BR-06, BR-09, BR-12, BR-14, 9.1, BR-02/03 tokens
22:00–26:00	5. The staff side	BR-16, BR-17
26:00–30:00	Buffer	Reviewer's own requests, questions, anything the Scribe flagged as missed

Roles

Role	Name	Owns
DRIVER	Amara	The keyboard. Pastes commands and clicks. Doesn't explain.
NARRATOR	Johan	Talks the reviewer through each step and ties it to the BR. Keeps time.
ARCHITECT	Aditya, Wali	Design and "where does it live" questions; Appendix A has the answers. Agree before the day which of you takes which rows of Appendix A, so you never talk over each other.
SCRIBE	Joseph, Will	**Joseph:** the reviewer's chosen order ID (act 3) and the act 4 baseline and final numbers. **Will:** ticks Appendix C as each BR is shown and notes reviewer requests and gaps, for the buffer.

Before the review

The day before

1 **Keep secrets out of the code.** The services read their secrets from environment variables. The values in `docker-compose.yml` are only development fallbacks. A git-ignored `.env` in the repo root holds real values. **Done 23 Oct**: it exists, and `git check-ignore -v .env` confirms git ignores it.

2 **Start the stack** with the demo settings. Market hours are switched off so orders fill whenever the review happens. The idle timeout is 5 minutes instead of 1, so Client A isn't signed out while the Driver types in the terminal.

```
source ~/lmj-demo/demo.sh
start_stack               # waits until all four services are UP
```

Always load `demo.sh` first: it sets the demo settings, and a `docker compose` command run without them recreates services with market hours back on and a 1-minute timeout.

3 **The test clients** (created through the UI; the client ID and account ID are the same number):

Who	Login	On 23 Oct
Client A	`Lebron@lemarket.com` / `Pass123!` Client and account ID **49**	About $79,800 cash; AAPL 7, ASML 6, AVGO 11, LRCX 5, NVDA 13, TD 5, TSM 20; 12 orders: 11 filled (9 buys, 2 sells) and 1 rejected (order 62, `STALE_QUOTE`, made 23 Oct with step 4).
Client B	`Bronny@lemarket.com` / `Pass123!` Client and account ID **50**	About $3,900 cash; BRK-A 5, CAT 5, CVX 2, INTC 1, KO 2, SHOP 5; 11 orders. No stock in common with A.
New user	Email _____ Password _____	**Don't create it.** It is registered live in act 1.

Rehearsals add orders and change these numbers. That's fine: act 4 compares the totals before and after, not against this table.

4 **Give Client A a rejected order** (done 23 Oct: order 62; keep these steps in case the database is rebuilt). A refused placement is never saved, so a REJECTED order has to come from execution. Pause execution, place the order, make the quotes stale, then resume. In a terminal with `demo.sh` loaded:

```
exec_pause
# as Client A in the browser: buy 1 share of any stock (it stays SUBMITTED)
feed_stale
exec_resume
# the order becomes REJECTED with STALE_QUOTE; then put the feed back:
feed_reset
```

The pending order: any restart resumes execution and fills an open order, so a pending order can't be kept from the day before. The one created live in act 4 (step 4.1) is the "still pending" order. Narrator: point this out when showing the history in act 2.

5 **Build the overnight report summary** (BR-16), so long-range reports read it. **Done 23 Oct:** the 22 Oct run is Ready and the summary covers 2023-10-22 to 2026-10-22; the 21 Oct run stays FAILED ("did not run"), which is a good talking point. Only if the database is ever rebuilt, run `run_overnight_now`. It clears yesterday's "did not run" record (the service never re-runs a night it has already recorded), runs the job now, shows the result and restores the normal schedule. Takes about a minute.

Then open Analyst, Activity by stock, and check it shows a few dozen trades across several stocks. The seed data already holds a trade history.

6 **Rehearse act 4 twice** with a stopwatch. It must fit in 8 minutes. Every test is undone by its own last command; Appendix B lists how to recover.

One hour before

7 **Set up the terminals.** In **each** of the three terminals, first type:

```
source ~/lmj-demo/demo.sh
demo_help                  # lists every command
health                     # four {"status":"UP"}
totals                     # prints Client A (account 49)
```

8 **Arrange the screens:**

Window	Contents
Chrome, normal window	Tab 1: Client A (`Lebron@lemarket.com`) on https://localhost:4200, Trade page. Tab 2: Client A's Orders page, **dragged out into its own window beside tab 1**, so the trade form and the orders list are on screen together. Tab 3: staff app https://localhost:4201 signed in as `ops@seed.lemarket.com` / `Pass123!`
Chrome, private window	Tab 1: Client B (`Bronny@lemarket.com`) on :4200. Tab 2: staff app :4201 as `analyst@seed.lemarket.com` / `Pass123!`
Chrome, Guest window	https://localhost:4200/register, empty, for the new user
Terminal 1	Commands, with `demo.sh` loaded
Terminal 2	`kafka_feed` : the live order event feed from Kafka (leave it running; Ctrl+C stops it)
Terminal 3	`logs` : the live log of orders-service and trading-service

9 **Run the readiness checklist** in Appendix D.

Act 0 · Opening 0:00–1:00

0.1 DRIVER As the Narrator starts, capture a session token that will be **expired** by act 4. It belongs to a seed client nobody else uses:

```
capture_old
```

```
captured at 10:00:12: 200 (valid now)
```

0.2 NARRATOR "*Everything today is live on this laptop. It runs four services, one PostgreSQL database and Kafka in Docker. As agreed with the instructor, the market is US stocks only and the stack is local. We'll show each requirement working normally first, then break things. Please pick any order, client or test you'd like, and we'll follow you.*"

Act 1 · A brand-new client 1:00–4:00

1.1 DRIVER Guest window, Register. Fill in the form with the new user's details, but type `password` as the password. BR-01

SHOW The password rules light up as unmet: 8+ characters, an uppercase letter, a lowercase letter, a special character.

NARRATOR "*The form shows the rules, but the server enforces the same ones: registration rules are checked on the server, so a client that skips the form is still refused.*"

1.2 DRIVER Fix the password and finish registration with an initial deposit. You land signed in. BR-01

NARRATOR "*The password is stored only as a hash. The session is a signed token in an HTTP-only cookie, so page scripts can't read it.*"

1.3 DRIVER Sign out, then sign back in with the same email and password. BR-01

NARRATOR "*That's a secure return sign-in. Repeated wrong passwords lock the account for 15 minutes; we won't trigger that now.*"

1.4 NARRATOR "*Sessions are time-limited and revocable. An idle session ends on its own: it warns 30 seconds before, then signs out. The timeout is configurable; it's 1 minute by default and 5 minutes today, so we aren't signed out while typing in the terminal. We'll prove expiry and revocation in act 4.*" BR-03

DRIVER Leave the Guest window open and untouched. About 4½ minutes later it shows the warning and then signs out. Narrator: point at it when it happens.

Act 2 · Trading as Client A 4:00–11:00

2.1 DRIVER Normal window, Client A, Portfolio. BR-10

SHOW Cash balance and holdings: quantity, average cost, current value.

NARRATOR "*This is Client A's current position, reflecting the last executed trade.*"

ARCHITECT If asked where it lives: cash in `accounts.cash_balance`, shares in `holdings`.

2.2 DRIVER Market page. BR-12

SHOW US stock quotes updating live, each with the time of its quote.

NARRATOR "*These are current quotes from our price simulator for the 51 US stocks. Every order is executed against the quote at that moment, and an order is refused if the quote is old or missing.*"

2.3 DRIVER Trade page, choose a stock A doesn't hold yet, Buy, switch to **dollar amount**, type `500`. BR-13 · BR-04

SHOW The indicative price with "Price as of …", the number of whole shares $500 buys, and the leftover.

NARRATOR "*Before submitting, the client sees an indicative price and exactly what they'll get. Clients can buy by shares or by amount; orders are whole shares, enforced on the server too.*"

2.4 NARRATOR Before the Driver submits: "*Watch the Orders window on the right, and the terminal below it.*" BR-07 · BR-08

DRIVER Submit. **Don't touch the Orders window.** Bring Terminal 2 (the Kafka feed) into view.

SHOW The order appears in the Orders window and turns FILLED on its own, about a second later. Terminal 2 prints a line at submission and a line at the fill, with the fill price, `quoteSource` and `quoteTime`.

NARRATOR "*No refresh: the server pushes status changes to the page; that's the Live badge. The terminal shows the same order's events as they happen. The fill price differs slightly from the price at submission, because the order was priced at the live quote at the moment it executed, and that quote's time is stored with it. We'll trace it in act 3.*"

Execution checks for new orders every second, so the fill is quick. If the reviewer wants to watch it more slowly, use the pause and resume in 4.1.

2.5 DRIVER Back to Portfolio. BR-09 · BR-10

SHOW Cash went down by the cost, and the new holding appears.

NARRATOR "*Cash, holdings and the permanent trade record were updated in one database transaction: all or nothing. We'll try to break that in act 4.*"

2.6 DRIVER Trade page, **Sell** 1 share of a stock A holds, Submit. It fills. BR-04

2.7 DRIVER Three refusals, one after another: BR-05

- Sell **more shares than A holds**, for example 8 AAPL when A holds 7: refused, not enough shares.
- Buy **CAVS**, the suspended stock: refused, not tradable.
- Buy by dollar amount for **more than A's cash**, for example $100,000: refused, not enough cash.

NARRATOR "*Every order is checked for cash, holdings and whether the stock can be traded before it's accepted, and cash already set aside for open orders counts too. A refused order is never saved.*"

ARCHITECT If asked about location rules: `seed_restricted@seed.lemarket.com` (`Pass123!`) lives in New Jersey. Buying `TSM` is refused with LOCATION_RESTRICTED; every other stock works.

2.8 DRIVER Orders page, history. BR-11

SHOW 10+ orders in time order: filled buys, filled sells, the rejected one with its reason, and today's.

NARRATOR "*The full history in time order, including the rejected order and why it was rejected. A pending order is made live in act 4, when we take execution down.*"

2.9 DRIVER Note one of A's order IDs. Switch to the private window (Client B). BR-02

SHOW B's portfolio: different cash and different stocks. Then, in B's tab, open `https://localhost:4200/api/v1/orders/<A's order ID>`


```
{"success":false,"error":"Access denied","code":"ACCOUNT_ACCESS_DENIED"}   (403)
```


NARRATOR "*Client B, signed in, asks for Client A's order directly and is refused. The answer is the same for an ID that doesn't exist, so IDs don't leak. Every endpoint needs a valid session except sign-in, registration and health.*"

2.10 DRIVER Normal window, Learn tab, then the simulator, for 30 seconds. BR-18

NARRATOR "*Our additional capability is Learn: lessons, a glossary and a practice simulator for new investors. It reuses our price history and places no real orders. We chose it over social trading and a mobile app because it serves first-time clients at low risk.*"

Act 3 · Trace a trade the reviewer picks 11:00–14:00

3.1 NARRATOR "*Please pick any order from Client A's history.*"

SCRIBE Write down the order ID: _____

3.2 DRIVER Normal window, tab 3 (staff app as Trading Ops), Trade search. Search for Client A (Lebron, client 49), then click the chosen order ID. BR-15

SHOW The timeline, oldest first: SUBMITTED, VALIDATED, RULE_CHECKED, ACCEPTED, PENDING, FILLED with the price and quote used, then SETTLED with the cash and holdings change.

NARRATOR "*This is the full lifecycle for audit or a dispute: when it was placed, what was checked, when it was accepted, the exact quote it was priced at, and what it did to cash and holdings. Each step was written in the same transaction as the change it describes.*"

3.3 DRIVER The same order straight from the database: BR-15 · BR-14

```
trace <ORDER_ID>
```

ARCHITECT "*The order is in the `orders` table, with the price, quote source and quote time. Each step is in `audit_log`, attributed to the client and timestamped. The settlement receipt is in `settlement_receipts`. It's all one PostgreSQL database, in the `db` container, on a Docker volume on this laptop.*"

3.4 DRIVER Try to change history, as the database **owner**: BR-14

```
try_change <ORDER_ID>
```

```
ERROR:  Trade records are immutable: UPDATE on orders refused
ERROR:  Audit records are immutable: DELETE on audit_log refused
```

NARRATOR "*Final orders and audit records can't be changed or deleted, not even by the database owner. The app's own login doesn't have the permission, and a trigger refuses it for everyone.*"

ARCHITECT Durability: "*A backup job takes a full `pg_dump` every night and keeps the last seven. The restore steps are in `database/README.md`.*" To show it: `backups`

This only works on a FILLED or REJECTED order. If the reviewer picked an open one, use another order.

Act 4 · Break it 14:00–22:00

The pattern for every test: take a before picture, run one command that breaks something, use the app normally, show nothing was damaged, then run one command that fixes it. The totals are taken **once at the start and once at the end**, not after each test.

4.0 DRIVER baseline

```
totals
```

SCRIBE Write down: cash _____ · filled_orders _____ · holdings _____

NARRATOR "*These are Client A's numbers now. We'll make two buys during the tests. At the end everything must match these numbers plus exactly those two.*"

4.1 · Execution goes down, then the service restarts BR-06 · 9.1 · ~2½ min

DRIVER Stop order execution:

```
exec_pause
```

```
{"enabled":false,"settlementAvailable":true}
```

DRIVER As Client A, **buy 1 share** of any stock. Orders page: it stays **SUBMITTED** and doesn't fill. SCRIBE note which stock.

NARRATOR "*The order was checked, saved and accepted, but the part that executes orders is down. The client's intent isn't lost; it's in the database.*"

```
last_order
```

DRIVER Restart the service that executes orders:

```
restart_orders            # prints "UP (8085)" when it is back
```

NARRATOR (during the wait: 7–20 s in testing) "*We've restarted the order service. Nothing about that order was held in memory.*"

SHOW The order turns FILLED on its own. If it hasn't within 10 s, click Orders again.

NARRATOR "*Execution came back and completed the pending order. Nothing was lost across the restart.*"

Tested 23 Oct: the order stayed SUBMITTED while paused and filled about 1 s after the service was back. The restart clears the pause, because it is held only in memory.

4.2 · A failed deployment, part 1 BR-14 · start it now, it fails within ~5 s

DRIVER Deploy a broken release: the order service with a wrong database password.

```
bad_deploy
```

NARRATOR "*That's a bad release, the same as a broken task definition: a configuration that can't start. While it fails, let's test the sessions.*"

4.3 · Tampered, expired and revoked tokens BR-02 · BR-03 · ~1½ min

DRIVER

```
tokens
```

```
expired  (captured 10:00:12, unused since): 401
valid    (fresh sign-in):                   200
tampered (one character changed in the middle): 401
revoked  (same valid token after signing out):  401
```

NARRATOR "*An expired token, a token with one character changed, and a token whose session was signed out are each refused, with no special handling. Every service checks the signature with the shared secret on every request.*"

ARCHITECT Where the expiry and secret are configured: see Appendix A.

If the reviewer wants to see it in the browser: Client B window, DevTools, Application, Cookies, edit one character **in the middle** of `jwt`, then refresh. B is sent to the login page. **Never edit the last character**: it can be accepted, for the harmless reason in Appendix A ("A tampered token was accepted"). If a reviewer does it, the Architect explains.

Tested 23 Oct: valid 200, middle character changed 401, payload character changed 401, signed out 401, expired after 5 idle minutes 401.

4.2 · The failed deployment, part 2: roll back

DRIVER

```
deploy_status
```

```
lemarketjames-orders-service-1 ... exited ... Exited (1)
password authentication failed for user "lemarket_app"
```

SHOW Client A's Portfolio still loads. Placing an order fails (the web app answers 502), and nothing is saved.

NARRATOR "*The release didn't start, so it wrote nothing. Clients can still see their accounts. Now we roll back to the last good configuration.*"

```
rollback                 # ~20 s, prints "UP (8085)"
totals
```

NARRATOR "*It's back, and the data is intact: the baseline plus the one buy from 4.1.*"

4.4 · Failure midway through a fill BR-09 · ~2 min

DRIVER Make Client A's settlements crash after writing, before committing:

```
fault_on
```

DRIVER As Client A, **buy 2 shares** of a stock. It stays PENDING. Terminal 3 (the log) shows the settlement failing and being retried.

```
totals
fault_check
```

```
cash, filled_orders and holdings: unchanged since 4.2
{"accountId": …, "failing": true, "failures": 3}
```

NARRATOR "*Each attempt wrote the cash change, the shares and the trade record, then crashed before committing. The database threw all three away together. Nothing is half-done, and the order is kept and retried.*"

```
fault_off
```

SHOW Within seconds the order turns FILLED without a refresh.

NARRATOR "*It settled exactly once, even though it was attempted several times. A settlement receipt per order makes a retry harmless.*"

4.5 · The price feed goes down BR-12 · ~1 min

DRIVER

```
feed_down
```

DRIVER As Client A, try to buy any stock.

SHOW The buy is refused at once: "Market price is unavailable. Please try again." (`PRICE_UNAVAILABLE`). The quotes on the Market page disappear within about 5 s.

NARRATOR "*Without a current quote we never guess a price. The order is refused, with a clear reason.*"

```
feed_reset
```

SHOW Quotes come back within about 5 s.

4.6 · The proof BR-09 · 9.1

DRIVER

```
totals
```

SCRIBE Compare with the 4.0 baseline: **filled_orders + 2**, cash down by exactly the two purchases, holdings up by 1 and 2 shares of the stocks bought.

NARRATOR "*A stopped executor, two restarts, a broken release, a crash in the middle of a fill and a price outage. The only change is the two buys we made, each recorded exactly once. Nothing was lost and nothing was doubled.*"

Act 5 · The staff side 22:00–26:00

5.1 DRIVER Private window, tab 2 (Analyst), Reports, **Activity by stock**, with a range covering the last few months. BR-16

SHOW Trades, shares and value per stock, including today's demo trades.

DRIVER Then **Trade activity by period**, by month and then by week.

NARRATOR "*Historical analysis by stock and by period. Past days come from a summary built overnight, and today comes from live data. That's why the trades we placed a few minutes ago already show up.*"

5.2 DRIVER Reports, **Overnight reports**, latest run (Ready), download the CSV and open it. BR-16

NARRATOR "*Every night at 2 a.m. New York time, outside trading hours, a job refreshes the summary and produces the monthly trading summary. Analysts download it here.*"

5.3 ARCHITECT The separation, in about 30 seconds: BR-16

```
reports
```

"*Reports run in their own service, with their own connection to the database: one connection, read-only, with a 5-second limit per query. Long-range reports read this nightly summary, not the live order tables. We measured it: with eight analysts running reports nonstop, order times moved by about 4 percent. Jenkins runs that load test on every build.*"

5.4 DRIVER Analyst dashboard. BR-17

SHOW The "What changed" strip with flagged insights, and the tiles for volume, most-active stocks and active-client trend.

NARRATOR "*Internal insights on trading volume, the most active stocks and client trends. Anything that moved 25 percent or more against the previous period is flagged.*"

5.5 DRIVER Normal window, tab 3 (Trading Ops), **Alerts**. BR-17

SHOW Alerts for large orders and refused settlements, with a link to each trade's timeline.

NARRATOR "*Every order event goes onto Kafka, and a separate consumer raises these alerts for Trading Ops. Trading never waits for it.*"

Buffer 26:00–30:00

SCRIBE Read out anything flagged as not fully shown, and give the Driver time to show it. NARRATOR If something is incomplete, say so plainly and say what we'd do next.

If the reviewer asks…	Go to
"Show the session timeout"	The Guest window from act 1 has signed itself out. Otherwise leave a window idle for 4½ minutes.
"Trace a different trade"	Repeat 3.2–3.3 with their order ID.
"Where is the secret / the expiry / the database?"	Appendix A
"Can the database be reached from outside?"	Appendix A, Network
"Run that failure again"	Each act 4 test stands on its own; run its commands again.
"What happens to a client in a restricted state?"	2.7, `seed_restricted` buying TSM

Appendix A · Architect's answers

Question	Answer
Where is the token expiry configured? BR-03	The idle timeout is `LMJ_SESSION_INACTIVITYTIMEOUT` in `docker-compose.yml`, read from `SESSION_INACTIVITY_TIMEOUT`: 1 minute by default, 5 minutes today. The maximum session length is `jwt.expiration-ms=3600000` (1 hour) in `services/auth-service/src/main/resources/application.properties`; refreshing never extends a session past it.
Where is the signing secret, and where is it stored? 9.3	It's configured as `jwt.secret=${JWT_SECRET}` in each service's `application.properties`, so its value comes from the environment, never from a committed file. Locally it's in `.env`, which git ignores (show `git check-ignore -v .env`). The values in the code are development-only fallbacks. In the cloud set-up it comes from AWS Secrets Manager (`deploy/aws/README.md`). The same applies to `INTERNAL_API_TOKEN`, which guards service-to-service calls.
Where does the database live? BR-14	One PostgreSQL 16 database in the `db` container, stored on a Docker volume (`docker volume ls | grep lemarket`). The schema is versioned as numbered SQL files in `database/schema`. Backups: a nightly `pg_dump` by the `db-backup` container, keeping the last 7. The restore steps are in `database/README.md`.
Which table holds what?	`orders`: each order, its status, price, quote source and time. `audit_log`: every lifecycle step. `settlement_receipts`: what each fill did to cash and holdings. `accounts`: cash. `holdings`: shares. `cash_movement_receipts`: deposits and withdrawals. `reporting_daily_trades`: the nightly report summary. `order_alerts`: surveillance alerts.
Is the database open to the internet? 9.3	No. Every port is bound to `127.0.0.1`, so only this laptop can connect. Show `docker compose ps` (the PORTS column says `127.0.0.1:…`) and `lsof -nP -iTCP:5433 -sTCP:LISTEN`. The database also accepts only the restricted app login, which can't change audit records. The web apps are HTTPS only.
Why is reporting separate when it's one database? BR-16	The requirement is that analysis never affects trading, and we measure that. (1) Reports run in their own service, which trading doesn't call and which calls nothing back. (2) They read through their own pool: one connection, read-only, 5-second query limit. (3) Long-range reports read the nightly summary, not the live orders. (4) In the load test, eight analysts running reports nonstop moved order times by about 4%; before our fixes it was 117%. A second database would be more to run for no gain at this scale. If we ever need one, the change stays inside the reporting service, because reports may read only the reporting view and the summary.
Why can't a fill be half-done? BR-09	Cash, holdings, the settlement receipt and the SETTLED audit event are written in one database transaction. A crash rolls back all of them. The receipt is keyed by order, so a retry after a crash settles once and never twice.
Why is execution separate from placement? BR-06	Placement saves and accepts the order and returns. A separate engine polls for open orders and executes them. If the engine is down, orders wait in the database and complete when it returns; nothing depends on memory.
A tampered token was accepted BR-02	Only if the **last** character was edited. Tokens are signed with HMAC-SHA512, a 512-bit signature written as 86 base64 characters. 512 isn't a multiple of 6, so the last character carries only 2 real bits and the decoder ignores the other 4. Some edits of that one character (Q to X, for example) therefore decode to the **identical** signature: the token itself hasn't changed. Any edit that changes the token's content, anywhere else, is refused (tested). Fix if wanted: also reject tokens whose signature isn't in canonical base64.
Why is there no gateway?	Each web app's Nginx routes the paths and every service validates the token itself. One less component to fail, with the same security check everywhere.

Appendix B · If something goes wrong

Symptom	Fix
Quotes are missing or frozen	`feed_reset`
Client A's orders stay PENDING	`fault_off`
No order fills at all	`exec_resume` . If orders show DELAYED instead, market hours are back on: `start_stack` (in a terminal with `demo.sh` loaded)
orders-service is down after 4.2	`rollback` (up to ~25 s)
"command not found" for `totals` , `fault_on` …	This terminal hasn't loaded them: `source ~/lmj-demo/demo.sh`
A browser window was signed out	Sign in again. That's the BR-03 timeout working, so say so.
`curl` to `/internal` answers 401	Load the commands again: `source ~/lmj-demo/demo.sh` (it reads `.env`).
Anything else	`all_clear` puts every test switch back and shows the health of all four services. If a service is down: `start_stack` .
Demo data damaged during a rehearsal	Restore last night's backup (`database/README.md` , "Backups and restore"), or rebuild with `docker compose down -v && docker compose up -d --build` and redo the day-before steps.

Appendix C · Requirement coverage (Scribe ticks during the demo)

BR	Requirement	Shown in	✓
BR-01	Client registration and secure return sign-in	1.1–1.3	
BR-02	Client can access only their own positions, cash and orders	2.9, 4.3	
BR-03	Time-limited, revocable signed-in sessions	1.4, 4.3	
BR-04	Buy and sell orders for supported instruments	2.3, 2.6	
BR-05	Validate cash, holdings and tradability before accepting	2.7	
BR-06	Record the order before separate execution; keep intent if execution fails	4.1	
BR-07	Order status updates without a manual refresh	2.4, 4.1, 4.4	
BR-08	Current quote at execution; fill or reject	2.4, 3.2	
BR-09	Holdings, cash and trade record updated atomically	2.5, 4.4, 4.6	
BR-10	Accurate holdings and cash after the last trade	2.1, 2.5	
BR-11	Chronological history of orders and fills	2.8	
BR-12	Current quotes (US equities, as agreed)	2.2, 4.5	
BR-13	Indicative price before submission	2.3	
BR-14	Permanent, durable, attributed records	3.3, 3.4, 4.2	
BR-15	Reconstruct a trade's full lifecycle	3.2, 3.3	
BR-16	Historical analysis independent of live trading	5.1–5.3	
BR-17	Insights: volumes, active stocks, client trends	5.4, 5.5	
BR-18	One additional capability, justified	2.10	
9.1	Survives restarts with nothing lost or doubled	4.1, 4.2, 4.6	
9.3	Secrets and network	Appendix A	

Reviewer requests and gaps:

Time	Request or gap	Covered later?

Appendix D · Readiness checklist

Do this the day before, and again one hour before the review.

Check	How	Day	−1 h
Stack running, healthy and reachable	Four `{"status":"UP"}`; https://localhost:4200 and :4201 load		
Client A and Client B have different cash and holdings	Both portfolios side by side		
Client A has 10+ orders, including a rejected one	Orders page; `totals`		
US stocks price correctly	Market page ticking; quote times current		
Two browser sessions plus a Guest window ready	Before the review, step 8		
Saved queries return the expected results	`totals`, `trace 62`, `reports`		
Every failure test rehearsed twice and undone	Act 4, timed under 8 minutes		
Reporting doesn't slow order placement	Last Jenkins load test green; the numbers in Appendix A		
Secrets in `.env`, not in git	`git check-ignore -v .env`		
Overnight summary built	Overnight reports page shows a Ready run		
Live feeds open	Terminal 2 on the Kafka order events; Terminal 3 following the orders-service and trading-service logs		
Market hours off, timeout 5 min	`demo.sh` loaded in every terminal before any `docker compose`		
Roles agreed; one full practice run in 30 minutes	Stopwatch		

From the guide: a requirement that can't be shown live is marked as not met, even if it was built. If something is incomplete, tell the reviewer before starting and say what you would do next.